

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Partea a IV-a
Finalizarea Proiectului

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Faza de Implementare	2
1.1	Arhitectura Sistemului și Comunicarea între Microservicii	2
1.2	Tehnologii Backend: Spring Boot	2
1.3	Tehnologii Frontend: JavaFX și FXML	2
1.4	Medii de Dezvoltare și Instrumente de Inginerie Software	3
1.5	Centralizarea Stivei Tehnologice	3
2	Șabloane de Proiectare (Design Patterns)	4
2.1	Șabloane Creaționale: Șablonul Singleton	4
2.1.1	Descriere și Aplicabilitate în Proiect	4
2.1.2	Avantaje și Justificare Tehnică	4
2.2	Șabloane Structurale: Șablonul Adaptor (Adapter / Wrapper)	4
2.2.1	Descriere și Aplicabilitate în Proiect	4
2.2.2	Avantaje și Justificare Tehnică	4
2.3	Șabloane Structurale: Șablonul Obiect de Categorie Ușoară (Flyweight)	5
2.3.1	Descriere și Aplicabilitate în Proiect	5
2.3.2	Avantaje și Justificare Tehnică	5
2.4	Șabloane Comportamentale: Șablonul Comandă (Command)	5
2.4.1	Descriere și Aplicabilitate în Proiect	5
2.4.2	Avantaje și Justificare Tehnică	5
2.5	Șabloane Comportamentale: Șablonul Strategie (Strategy Pattern)	5
2.5.1	Descriere și Aplicabilitate în Proiect	5
2.5.2	Avantaje și Justificare Tehnică	6
3	Descrierea aplicației	7
3.1	Interfața Utilizatorului Neautentificat	7
3.2	Interfața Utilizatorului Angajat	8
3.3	Interfața Utilizatorului Manager	11
3.4	Interfața Utilizatorului Administrator	13

1 Faza de Implementare

Realizarea sistemului informatic pentru agenția de rezervare a biletelor de avion impune utilizarea unei stive tehnologice moderne, capabile să susțină o arhitectură distribuită, modulară și scalabilă. În conformitate cu cerințele de proiectare, aplicația utilizează o arhitectură bazată pe microservicii, separând complet logica de business de interfața grafică cu utilizatorul (Client UI). În acest capitol sunt detaliate și justificate deciziile tehnologice fundamentale, mediile de dezvoltare și instrumentele software folosite pe parcursul etapelor de analiză, proiectare și implementare.

1.1 Arhitectura Sistemului și Comunicarea între Microservicii

Sistemul este structurat în **6 microservicii independente** și o componentă de **API Gateway**. Această abordare oferă o izolare completă a contextelor delimitate (Bounded Contexts), respectând principiile *Domain-Driven Design* (DDD).

- **Cele 6 Microservicii:** Fiecare microserviciu încapsulează o funcționalitate distinctă a agenției. Această segmentare permite scalarea individuală și mentenanța facilă a fiecărui modul.
- **API Gateway:** Acționează ca punct unic de intrare pentru aplicația client (JavaFX). Acesta are rolul de a ruta cererile HTTP către microserviciile corespunzătoare, de a centraliza politicile de securitate (verificarea token-urilor de autentificare) și de a ascunde topologia internă a rețelei de microservicii, sporind astfel securitatea și performanța.
- **Justificarea Comunicării:** Comunicarea dintre aplicația client și API Gateway, precum și comunicarea inter-servicii, se realizează în mod sincron prin intermediul protocolului **HTTP/REST (Representational State Transfer)**. Datele sunt serializate în format **JSON**, asigurând un overhead minim și o interoperabilitate maximă.

1.2 Tehnologii Backend: Spring Boot

Pentru dezvoltarea celor 6 microservicii și a API Gateway-ului s-a utilizat framework-ul **Spring Boot** (bazat pe limbajul de programare Java).

- **Motivele alegerii:** Spring Boot reduce drastic timpul de configurare inițială prin paradigma *Convention over Configuration*. Acesta oferă un server web încorporat (Apache Tomcat), facilitând lansarea independentă a fiecărui microserviciu ca un proces de sine stătător.
- **Persistența Datelor:** S-a utilizat ecosistemul **Spring Data JPA** împreună cu implementarea **Hibernate** ca motor de ORM (Object-Relational Mapping). Această tehnologie permite maparea claselor de model (entităților DDD) direct pe tabelele din baza de date relațională, eliminând necesitatea scrierii manuale a interogărilor SQL complexe pentru operațiile de bază și garantând consistența tranzacțională a datelor (esențială în cazul rezervării locurilor în avion).
- **Securitate și Autorizare:** Prin utilizarea modulelor Spring, se realizează controlul accesului bazat pe roluri (*Role-Based Access Control - RBAC*), asigurând izolarea strictă a operațiunilor permise pentru **Angajat**, **Manager** și **Administrator**, conform specificațiilor problemei.

1.3 Tehnologii Frontend: JavaFX și FXML

Interfața grafică a aplicației client este dezvoltată utilizând **JavaFX**, o platformă avansată pentru crearea de aplicații de tip Desktop bogate în elemente vizuale (Rich Internet Applications).

- **Separarea Prezentării (FXML):** S-a utilizat limbajul bazat pe XML numit **FXML** pentru definirea structurii declarative a interfețelor grafice. Acest lucru permite o separare clară între design-ul vizual (fișierele `.fxml`) și logica de control din spate (clasele `ModelView` din Java), respectând șablonul de proiectare arhitectural MVVM (Model-View-ViewModel).
- **Suport pentru Internaționalizare (i18n):** Pentru a îndeplini cerința ca interfața grafică să fie disponibilă în cel puțin 3 limbi de circulație internațională, s-a apelat la clasa `ResourceBundle` din Java. Textele din aplicație nu sunt hardcodate, ci sunt încărcate dinamic din fișiere de configurare de tip `Properties` (ex: `messages_ro.properties`, `messages_en.properties`, `messages_de.properties`), în funcție de preferința utilizatorului.

- **Generare de Grafice și Export:** JavaFX oferă suport nativ excelent pentru componentele de tip `ChartView` (`PieChart`, `BarChart`), utilizate pentru a afișa cele minimum 3 statistici vizuale solicitate de utilizatorul de tip `Manager`. De asemenea, s-au integrat librării Java software complementare (precum Apache POI sau iText/OpenPDF) pentru a permite descărcarea listelor și biletelor în formatele cerute: **PDF, CSV, JSON, XML și DOC**.

1.4 Medii de Dezvoltare și Instrumente de Inginerie Software

Pe parcursul ciclului de viață al dezvoltării proiectului, au fost utilizate următoarele instrumente corelate:

1. **IntelliJ IDEA:** Ales ca Mediu de Dezvoltare Integrat (IDE) principal pentru scrierea codului sursă Java/JavaFX și Spring Boot. Oferă instrumente avansate de refactoring, analiză statică a codului, debugging distribuit și suport integrat pentru ecosistemul Spring, crescând productivitatea și reducând apariția erorilor de sintaxă sau logică.
2. **Apache Maven:** Utilizat pentru managementul dependențelor și automatizarea procesului de build (*Build Automation Tool*). Fiecare dintre cele 6 microservicii, API Gateway-ul și proiectul Client dețin propriul fișier `pom.xml`. Maven asigură descărcarea automată a librăriilor necesare (Spring Starter, drivere de bază de date, componente JavaFX) și garantează reproductibilitatea compilării proiectului pe orice mașină de calcul.
3. **DataGrip:** Instrumentul de administrare a bazelor de date dezvoltat de JetBrains, utilizat pentru scrierea, testarea și optimizarea structurilor SQL. Într-o arhitectură de microservicii, unde fiecare serviciu își gestionează propria persistență (bază de date per serviciu), DataGrip a facilitat conectarea simultană la mai multe scheme de date, permițând vizualizarea rapidă a tabelor de zboruri, utilizatori și rezervări.
4. **Swagger / OpenAPI:** Utilizat pentru documentarea și testarea interfețelor API expuse de microservicii. Integrarea Swagger în Spring Boot generează automat o pagină web interactivă pentru fiecare microserviciu, permițând echipei de dezvoltare să testeze endpoint-urile (cererile de GET, POST, PUT, DELETE pentru zboruri și bilete) direct din browser, fără a fi necesară pornirea completă a clientului JavaFX în fazele incipiente.
5. **Draw.io:** Instrumentul utilizat pentru realizarea tuturor diagramelor UML solicitate în fazele de analiză și proiectare. Cu ajutorul acestuia au fost construite precis: diagrama cazurilor de utilizare, diagramele de activități corelate, diagrama de pachete, diagramele de clase structurate pe principii DDD, diagramele entitate-relație (ERD) pentru baze de date, diagramele de secvență pentru fluxurile operaționale, precum și diagramele de componente și dezvoltare (deployment) pentru topologia microserviciilor.

1.5 Centralizarea Stivei Tehnologice

În tabelul 1 este prezentată o sinteză a corelării dintre componentele arhitecturale și tehnologiile sau instrumentele utilizate.

Tabela 1: Sinteza stivei tehnologice utilizate în proiect

Componentă / Etapă	Tehnologie / Instrument	Rol în cadrul Sistemului
Interfață Client (Frontend)	JavaFX, FXML, Properties (i18n)	UI multi-lingv, grafice statistice, exporturi
Logică Business (Backend)	Java, Spring Boot	Implementarea celor 6 microservicii specifice
Poartă de Acces	Spring Cloud Gateway / API GW	Rutare cereri client, securitate centralizată
Persistența Datelor	Spring Data JPA, Hibernate, SQL	Managementul tranzacțional al rezervărilor
Management Proiect	Apache Maven	Gestiunea dependențelor și automatizare build
Medii de Lucru (IDEs)	IntelliJ IDEA, DataGrip	Dezvoltare cod sursă și administrare DB
Documentare API	Swagger / OpenAPI	Interfață de testare și expunere endpoint-uri
Modelare Grafică	Draw.io	Generarea diagramelor UML (Analiză/Proiectare)

2 Șabloane de Proiectare (Design Patterns)

În vederea asigurării unei arhitecturi scalabile, flexibile și ușor de întreținut, atât în cadrul aplicației client (dezvoltată în JavaFX), cât și în ecosistemul de microservicii din backend (Spring Boot), au fost implementate 5 șabloane de proiectare fundamentale din catalogul Gang of Four (GoF). Acestea sunt clasificate în cele trei categorii consacrate: creaționale, structurale și comportamentale.

2.1 Șabloane Creaționale: Șablonul Singleton

2.1.1 Descriere și Aplicabilitate în Proiect

Șablonul *Singleton* garantează că o clasă are o singură instanță globală pe parcursul ciclului de viață al aplicației și oferă un punct unic de acces la aceasta. În cadrul acestui proiect, șablonul este utilizat în două moduri complementare:

- **Aplicația Client (Frontend):** Clasa `LanguageManager` este proiectată ca o instanță unică globală care gestionează localizarea și internaționalizarea întregii interfețe grafice, oferind acces static la pachetul de resurse active (`ResourceBundle`) pentru traducerea dinamică în cele 3 limbi de circulație internațională.
- **Microservicii (Backend):** Managementul instanțelor pentru componentele arhitecturale din backend (precum `ZboruriService`, `BiletService`, `UtilizatoriService`) este delegat containerului Inversion of Control (IoC) din framework-ul Spring. Toate clasele adnotate cu `@Service`, `@RestController` sau `@Component` sunt instanțiate implicit ca un *Singleton Bean* la pornirea sistemului.

2.1.2 Avantaje și Justificare Tehnică

- **Optimizarea Resurselor de Memorie:** Prin eliminarea instanțierilor repetate ale claselor de servicii și de management, se reduce considerabil amprenta de memorie și se diminuează presiunea asupra mecanismului de Garbage Collection.
- **Siguranță în Mediu Concurent (Thread-Safety):** Deoarece serviciile din backend sunt complet *stateless* (nu rețin date de stare specifice unui anumit client în atributele lor), instanța unică poate procesa în siguranță sute de cereri HTTP concurente transmise simultan de aplicația client.
- **Consistență Globală:** Pe componenta de frontend, utilizarea unui punct unic de control pentru limbă previne desincronizarea traducerilor între diferitele ferestre FXML deschise concomitent.

2.2 Șabloane Structurale: Șablonul Adaptor (Adapter / Wrapper)

2.2.1 Descriere și Aplicabilitate în Proiect

Șablonul *Adapter* convertește interfața unei clase într-o altă interfață așteptată de clienți, permițând colaborarea unor componente care altfel ar fi incompatibile din cauza interfețelor diferite. Acest pattern reprezintă pilonul de integrare în arhitectura hexagonală a microserviciilor din backend:

- **Adaptarea Persistenței (DAO):** Clasele de tip Data Access Object (ex. `BiletDAO`, `UtilizatorDAO`, `NotificareDAO`) acționează ca adaptorii peste repository-uriile Spring Data JPA. Ele traduc operațiile specifice bazei de date în concepte pure de domeniu economic.
- **Adaptarea Comunicației Inter-Service:** Pentru apelurile REST efectuate între microservicii (de exemplu, când microserviciul de *Statistici* sau cel de *Bilete* interoghează microserviciul de *Zboruri*), se utilizează adaptorii dedicați de rețea precum `ZborServiceAdapter` sau `BiletClientAdapter`. Aceștia încapsulează logica apelurilor HTTP (efectuate prin `RestTemplate` sau `WebClient`), serializarea/deserializarea JSON și gestionarea erorilor de conexiune.

2.2.2 Avantaje și Justificare Tehnică

- **Decuplarea Logicii de Business:** Nucleul aplicației (stratul *Domain*) nu are cunoștință despre detaliile tehnologice de infrastructură (cum ar fi tabelele SQL sau protocolul REST), interacționând doar cu Porturi abstracte.
- **Independență Tehnologică și Mentenabilitate:** Dacă în viitor protocolul de comunicație inter-service se schimbă de la REST (HTTP) la gRPC, sau dacă baza de date SQL este înlocuită cu una NoSQL, modificările se vor rezuma strict la nivelul adaptorului extern, lăsând logica de business complet intactă.

2.3 Șabloane Structurale: Șablonul Obiect de Categorie Ușoară (Flyweight)

2.3.1 Descriere și Aplicabilitate în Proiect

Șablonul *Flyweight* vizează optimizarea utilizării memoriei prin partajarea extinsă a stărilor comune (intrinseci) ale unui volum mare de obiecte similare, evitând instanțierea redundantă.

- **Aplicabilitate în Proiect:** Acest șablon este localizat în aplicația client, în cadrul componentei `FlightSeatMapViewMo` responsabilă de randarea matricei interactive a locurilor din avion (structură de 15 rânduri pe 6 coloane). Pentru a nu alocă în memorie sute de obiecte JavaFX de tip `SimpleStringProperty` sau `SimpleBooleanProperty` pentru modificările individuale de stil sau blocare aferente fiecărui scaun, acestea sunt stocate într-o structură de tip `Cache` (`Map<String, StringProperty> seatStyles`). Metoda `seatStyleProperty(seatName)` returnează proprietatea existentă sau o creează dinamic doar la prima solicitare.

2.3.2 Avantaje și Justificare Tehnică

- **Reducerea Masivă a Amprentei de Memorie pe Client:** Împiedică supraîncărcarea memoriei RAM a aplicației desktop prin re folosirea proprietăților de stilizare JavaFX.
- **Fluidizarea Randării Interfeței Grafice (UI Performance):** Prin reducerea numărului total de obiecte active ascultate de motorul JavaFX, operațiunea de redesenare a hărții scaunelor (atunci când un loc devine ocupat sau liber în urma unei vânzări/anulări) se execută instantaneu, asigurând o experiență de utilizare fluidă.

2.4 Șabloane Comportamentale: Șablonul Comandă (Command)

2.4.1 Descriere și Aplicabilitate în Proiect

Șablonul *Command* încapsulează o cerere sau o acțiune a utilizatorului sub forma unui obiect independent, permițând parametrizarea clienților cu diverse cereri și decuplarea completă a obiectului care invocă acțiunea de cel care o execută.

- **Aplicabilitate în Proiect:** Reprezintă coloana vertebrală a logicii comportamentale din aplicația client JavaFX. Toate acțiunile declanșate din interfața grafică implementează o interfață comună `ICommand` (ce expune metoda `execute()`). Exemple concrete includ: `LoginCommand`, `SaveUserCommand`, `DeleteBiletCommand`, `ExportCsvCommand` și comenzile de navigare prin ecrane (`NavZboruriCommand`). Controlerul vizual (`View`) doar apelează `command.execute()` la interacțiunea cu utilizatorul, fără a cunoaște logica internă.

2.4.2 Avantaje și Justificare Tehnică

- **Decuplare Arhitecturală Totală:** Clasele din stratul `View` (UI) și proprietățile reactive din `ViewModel` nu conțin cod pentru procesarea efectivă a datelor sau apeluri HTTP, fiind complet izolate de logica de business.
- **Facilitarea Procesării Asincrone (Non-blocking UI):** Încapsularea sub formă de obiecte comandă permite delegarea facilă a execuției pe fire de fundal secundare (Background Threads) prin `CompletableFuture.runAsync` prevenind blocarea sau înghețarea interfeței grafice în timpul descărcării rapoartelor mari sau interogărilor de rețea.

2.5 Șabloane Comportamentale: Șablonul Strategie (Strategy Pattern)

2.5.1 Descriere și Aplicabilitate în Proiect

Șablonul *Strategy* definește o familie de algoritmi, îi încapsulează pe fiecare într-o clasă separată și îi face complet interschimbabili în timpul rulării, permițând algoritmului să varieze independent de clienții care îl utilizează.

- **Aplicabilitate în Proiect:** Acest șablon este utilizat în arhitectura microserviciului de *Export* pentru generarea fișierelor solicitate din interfață. Se definește o interfață comună `IExportStrategy`, iar formatele specifice cerute în enunțul proiectului (CSV, JSON, XML, PDF) sunt implementate ca strategii concrete separate (ex. `CsvExportStrategy`, `PdfExportStrategy`). Serviciul central `ExportService` interacționează exclusiv cu interfața abstractă, selectând strategia potrivită la rulare pe baza parametrului primit de la client.

2.5.2 Avantaje și Justificare Tehnică

- **Respectarea Principiului Open-Closed (SOLID):** Structura codului devine extensibilă. Dacă în viitor agenția de aviație solicită adăugarea unui nou format de raportare (cum ar fi Microsoft Excel - `.xlsx`), integrarea se realizează prin simpla scriere a unei noi clase ce implementează `IExportStrategy`, fără a modifica controllerul sau logica existentă a serviciului central.
- **Eliminarea Blocurilor Condiționale Complexe:** Evită utilizarea structurilor repetitive de tipul `if-else` sau `switch` masive pentru determinarea algoritmului de parsare, îmbunătățind lizibilitatea și testabilitatea unitară a codului sursă.

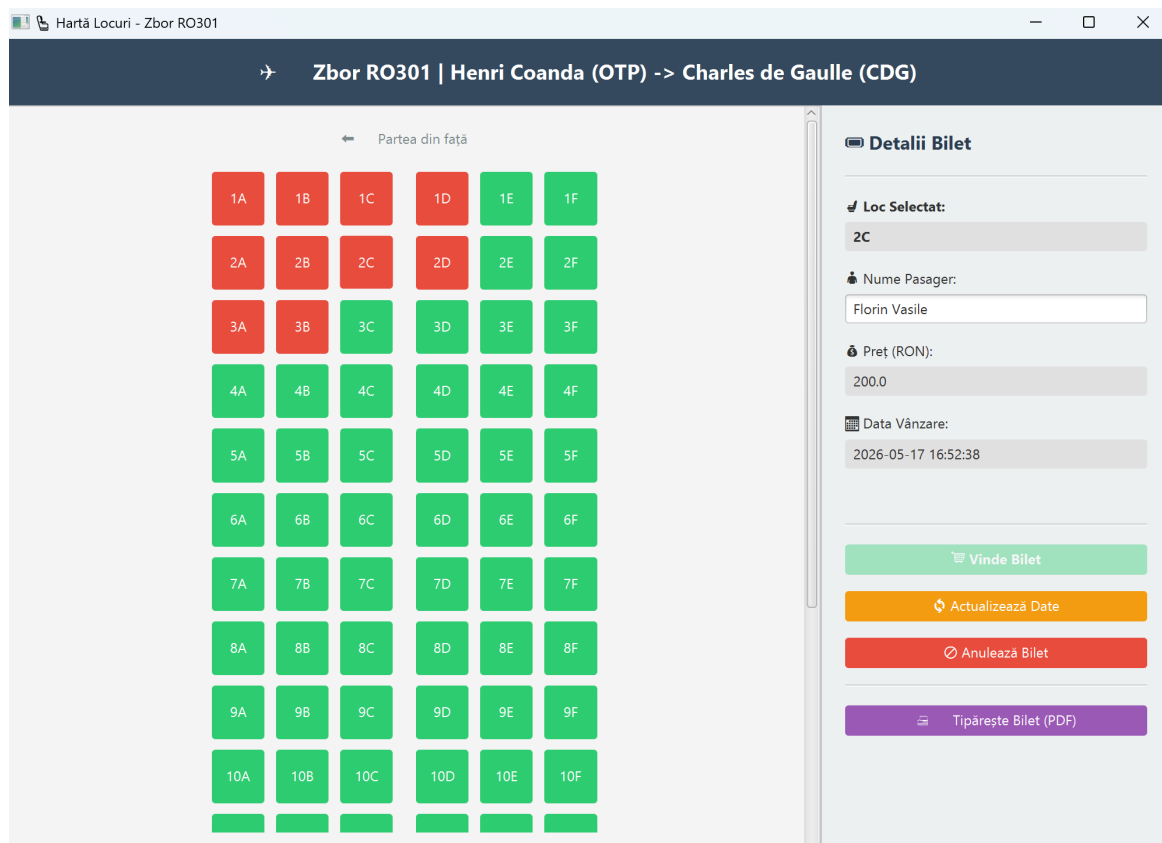


Figura 4: Vizualizarea interactivă a hărții locurilor din avion

Harta interactivă a locurilor: Aceasta este probabil cea mai complexă componentă vizuală a aplicației. Layout-ul este împărțit în două zone: în stânga, o reprezentare vizuală a dispunerii scaunelor în aeronavă (utilizând un cod de culori sugestiv - verde pentru liber, roșu pentru ocupat); în dreapta, formularul dinamic de detalii ale biletului. Interacțiunea cu un scaun actualizează instantaneu formularul lateral. Butoa-nele de acțiune inferioare (Vinde, Actualizează, Anulează, Tipărește) își modifică starea (activat/dezactivat) în mod inteligent, în funcție de contextul scaunului selectat, prevenind astfel erorile umane de operare.

3.3 Interfața Utilizatorului Manager

Interfața destinată managerului este concepută pentru administrarea bazei de date a zborurilor și analiza performanțelor agenției. Pentru a demonstra capabilitățile de internaționalizare (i18n), capturile au fost realizate având selectată limba germană.

Manager - Maria Ionescu Abmelden Sprache: Deutsch

Manager-Panel
Flugverwaltung
Grafische Statistiken

Abflug: Ankunft: Datum: 5/19/2026 Suchen Alle Flüge Flugnummer: Ex: RO321 Filtern
Exportformat: CSV Flüge Exportieren

Flugdatensatz-Formular

Flugnummer: RO301 Abflughafen: Henri Coanda (OTP) Ankunftsflughafen: Charles de Gaulle (CDG)
Flugdatum: 6/1/2026 Abflugzeit (HH:mm): 11:00 Ankunftszeit (HH:mm): 13:30
Ticketpreis (RON): 200.0 Verfügbare Plätze: 80
Neuer Flug Flug löschen Änderungen speichern

Flugnumm...	Abflughafen:	Ankunftsflughafen:	Abflugzeit (HH:mm)	Ankunftszeit (HH:mm)	Ticketpreis (R...	Verfügbare Pl...
RO301	Henri Coanda (OTP)	Charles de Gaulle (...)	2026-06-01 11:00	2026-06-01 13:30	200.0	80
W63301	Avram Iancu (CLJ)	Charles de Gaulle (...)	2026-06-02 12:15	2026-06-02 14:00	100.0	75
BA102	Henri Coanda (OTP)	Heathrow (LHR)	2026-06-03 17:00	2026-06-03 18:30	300.0	0
LH142	Frankfurt Airport (...)	Heathrow (LHR)	2026-06-04 14:00	2026-06-04 15:45	250.0	86
RO505	Charles de Gaulle (...)	Henri Coanda (OTP)	2026-06-05 21:00	2026-06-06 00:00	150.0	90
AZ220	Heathrow (LHR)	Leonardo da Vinci (...)	2026-06-06 10:30	2026-06-06 13:00	500.0	89
IB303	Henri Coanda (OTP)	Barajas (MAD)	2026-06-07 22:20	2026-06-08 01:15	120.0	1
RO999	Avram Iancu (CLJ)	Frankfurt Airport (...)	2026-06-08 16:00	2026-06-08 17:10	80.0	45

Flug ausgewählt.

Figura 5: Panoul Managerului - Gestiunea zborurilor (Limba Germană)

Gestiunea Zborurilor: Ecranul „Flugverwaltung” combină un panou de filtrare flexibil, un formular de introducere a datelor extrem de detaliat și un tabel de previzualizare. Managerul poate realiza operațiuni CRUD (Creare, Citire, Actualizare, Ștergere) direct din această fereastră. Formularul conține controale avansate precum selecții derulante pentru aeroporturi și selectoare de date (**DatePicker**), asigurând astfel introducerea standardizată a datelor în sistem.

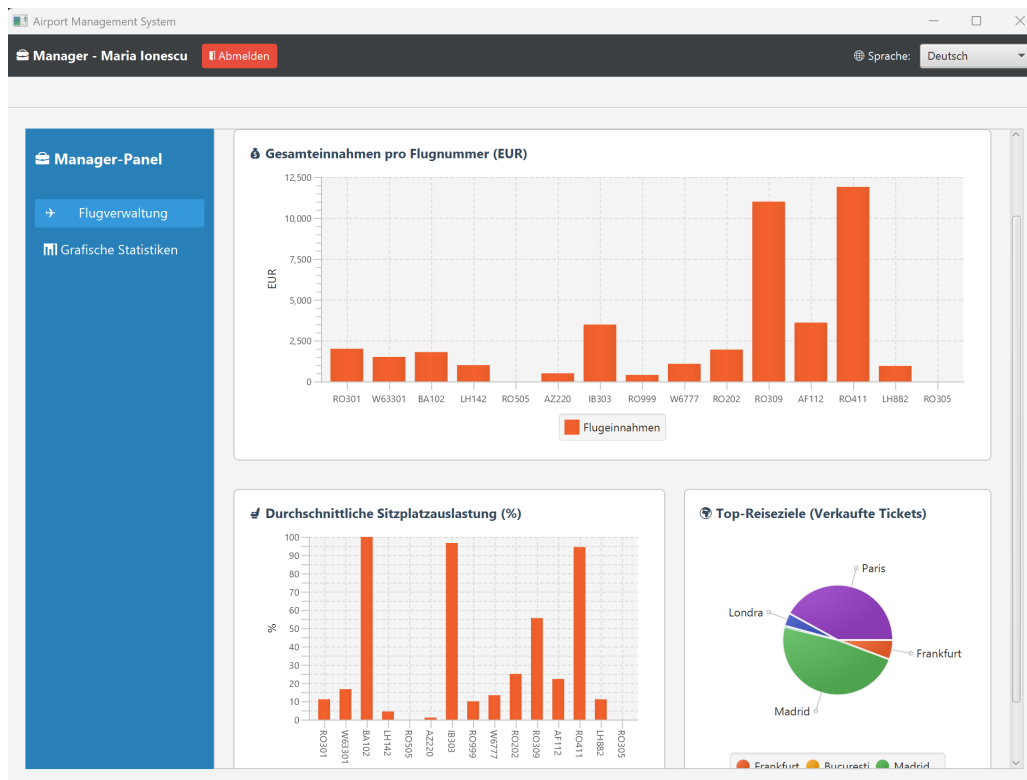


Figura 6: Panoul Managerului - Dashboard Statistici (Limba Germană)

Grafice și Statistici: Ecranul „Grafische Statistiken” ilustrează un dashboard decizional. S-au integrat vizualizări de tip BarChart pentru urmărirea veniturilor pe zbor și a gradului mediu de ocupare al aeronavelor. De asemenea, s-a utilizat un PieChart pentru a evidenția vizual topul celor mai vândute destinații (ex. Paris, Frankfurt). Aceste componente transformă datele brute într-o formă ușor de asimilat, ajutând managementul în luarea deciziilor strategice.

3.4 Interfața Utilizatorului Administrator

Aplicația oferă administratorului unelte de securitate și gestiune a personalului, interfața demonstrând aici suportul nativ pentru limba engleză.

The screenshot displays the 'Administrator Panel' of the 'Airport Management System'. The interface is in English. On the left is a dark sidebar with a 'Users' menu item. The main content area is titled 'User Details (Selection / New)' and contains a form for adding or editing a user. The form fields are: Full Name (Maria Ionescu), Email (manager@aeroport.ro), Phone (0740333444), Password (manager123), and Role (Manager). Below the form is a 'Filter by Role' dropdown and an 'Export CSV' button. A table lists the system's users, with the second row (Maria Ionescu) highlighted in blue. At the bottom, a status bar shows 'User selected.' and buttons for 'New Form', 'Delete', and 'Save'.

Full Name	Email	Password	Phone	Role	Creation Date
Ion Popescu	admin@aeroport.ro	admin123	0740111222	Administrator	2026-05-16 21:35
Maria Ionescu	manager@aeroport.ro	manager123	0740333444	Manager	2026-05-16 21:35
Andrei Radu	angajat1@aeroport.ro	angajat123	0722555666	Employee	2026-05-16 21:35
Elena Vasilescu	angajat2@aeroport.ro	angajat123	0722777888	Employee	2026-05-16 21:35
Alexandru Nita	angajat3@aeroport.ro	angajat123	0733888999	Employee	2026-05-16 21:35
Diana Oprea	angajat4@aeroport.ro	angajat123	0755444333	Employee	2026-05-16 21:35

Figura 7: Panoul Administratorului - Gestionarea conturilor de utilizator

Gestiunea Utilizatorilor: Submeniul „Users” afișează un formular dedicat prelucrării datelor de personal. Administratorul poate crea conturi noi de tip Manager sau Angajat (Employee) și poate gestiona drepturile de acces. Zona inferioară prezintă tabelul complet al personalului, cu posibilitatea de filtrare rapidă după rol (Filter by Role) și de export al datelor într-un raport CSV, esențial pentru auditul de securitate. Modificările operate aici declanșează fluxul automat de alertare a utilizatorilor, administrat de arhitectura backend.

